Robot Tutorial

Henrik Christensen, Charvi Shukla, Shiladitya Biswas
Credit to Chenghao Li, Henry Zhang, David Paz for Initial version

Contextual Robotics Institute | Autonomous Vehicle Lab

UC San Diego
JACOBS SCHOOL OF ENGINEERING

Overview

- System Overview
- UART connection and WiFi connection
- ROS2 packages
- Control for the robot
- Camera Driver
- Apriltag Detection

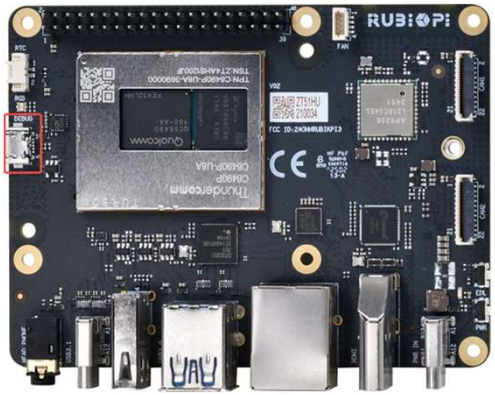
Waverover System Overview



- Robot body: Waverover
  - 4 Wheels Differential Drive
  - Battery
  - Raspberry Pi Camera
  - IMU
- Robot Head: Qualcomm Rubik Pi
  - Kryo™ 670 8 cores CPU
  - RAM 8 GB LPDDR4x
  - 12 TOPS On-device AI capability
- Robot Brain: Software
  - System: Ubuntu 24.04
  - ROS2 Jazzy
  - Custom ROS2 Packages: robot_vision_camera, robot_contrl, apriltag_ros

Setup Debug UART (link)

- Connect Rubik Pi through micro-usb cable.
  - Install screen/minicom for ubuntu and MacOS Or PuTTY for Windows



```
$ sudo apt update
```

```
$ sudo apt install screen
```

```
$ ls /dev/ttyACM*
```

```
$ sudo screen /dev/ttyACM* 115200 or sudo minicom -D /dev/ttyACM* -b 115200
```

  - Account: ubuntu
  - Password: rubikpi123
  - passwd

WiFi connection

- Set-up WiFi connection through terminal (link)
  - nmcli

```
$ nmcli dev wifi connect <WiFi-SSID> password <WiFi-password>
```

  - nmtui

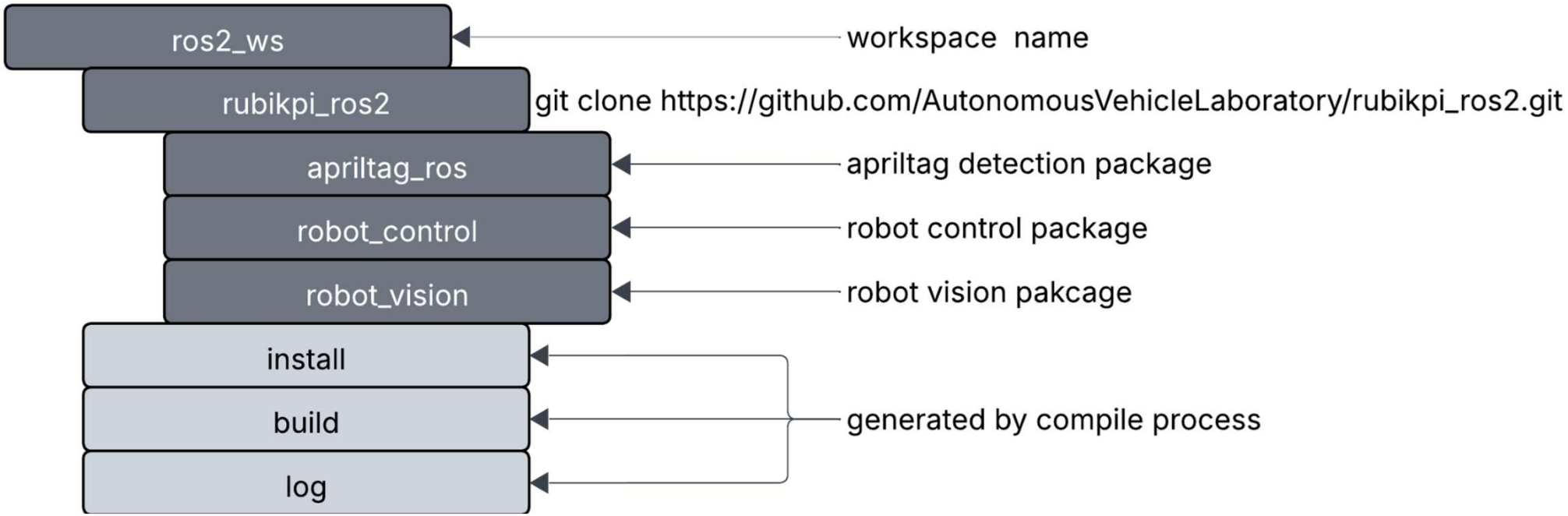
```
$ sudo nmtui
```

- Access over SSH (link)

```
$ ssh <user name>@ip-address
```

```
$ ssh ubuntu@192.168.90.181
```

Custom ROS2 Packages



Robot Control of Waverover (link)

- Connect to the robot's WiFi network
  - ssid: UGV
  - password: 12345678
- Access the web-based user interface
  - open the Google Chrome browser and type "192.168.4.1" in the address bar
- Control robot using JSON
  - `{"T":1,"L":0.5,"R":0.5}`
  - L is the speed of the left wheel, R is the speed of the right wheel, and the speed range is -0.5 ~ +0.5, positive value forward, negative value backward.
  - `{"T":126}`
  - get IMU data, including heading angle, geomagnetic field, acceleration, attitudes, temperature, etc.
  - `{"T":3,"lineNum":0,"Text":"putYourTextHere"}`
  - OLED screen display content settings.
  - lineNum parameter for the line settings, can be: 0, 1, 2, 3, a total of 4 lines of content can be displayed.



Robot Control of Waverover (ROS2)

- Motor control in ros2 using serial port /dev/ttyUSB0 (link)
- Add ROS2 Jazzy to your path

```
$ source /opt/ros/jazzy/setup.bash
```

- Build ROS2 packages

```
$ cd /path/to/ros2_ws/
```

```
$ colcon build
```

```
$ colcon build -packages-select package_name
```

```
$ colcon build -symlink-install
```

- Add workspace to your path

```
$ source /path/to/ros2_ws/install/setup.bash
```

Robot Control of Waverover (ROS2)

- Check ROS2 Domain ID:

```
$ echo $ROS_DOMAIN_ID
```

- Run nodes (link)

```
$ ros2 run robot_control motor_control #waverover motors node
```

```
$ ros2 run robot_control keyboard_control #keyboard control node
```

- Or (link)

```
$ ros2 launch robot_control robot_teleop_launch.py #keyboard control launch file
```

```
$ ros2 topic pub --once /motor_commands std_msgs/msg/Float32MultiArray "data: [0.2, 0.2]"
```

Organization of your workspace

tree -L 2

```
ubuntu@ubuntu:~/ros2_ws$ tree -L 2
.
├── build
│   ├── apriltag_ros
│   ├── COLCON_IGNORE
│   ├── robot_control
│   └── robot_vision_camera
├── install
│   ├── apriltag_ros
│   ├── COLCON_IGNORE
│   ├── local_setup.bash
│   ├── local_setup.ps1
│   ├── local_setup.sh
│   ├── _local_setup_util_ps1.py
│   ├── _local_setup_util_sh.py
│   ├── local_setup.zsh
│   ├── robot_control
│   ├── robot_vision_camera
│   ├── setup.bash
│   ├── setup.ps1
│   ├── setup.sh
│   └── setup.zsh
├── log
│   ├── build_2025-10-07_00-11-30
│   ├── COLCON_IGNORE
│   ├── latest -> latest_build
│   └── latest_build -> build_2025-10-07_00-11-30
└── rubikpi_ros2
    ├── apriltag_ros
    ├── robot_control
    └── robot_vision
```

```
ubuntu@ubuntu:~/ros2_ws/rubikpi_ros2$ tree -L 3
.
├── apriltag_ros
│   ├── cfg
│   │   └── tags_36h11.yaml
│   ├── CMakeLists.txt
│   ├── launch
│   │   ├── apriltag_launch.py
│   │   └── camera_36h11.launch.yml
│   ├── LICENSE
│   ├── package.xml
│   ├── README.md
│   └── src
│       ├── AprilTagNode.cpp
│       ├── conversion.cpp
│       ├── pose_estimation.cpp
│       ├── pose_estimation.hpp
│       ├── tag_functions.cpp
│       └── tag_functions.hpp
├── robot_control
│   ├── launch
│   │   └── robot_teleop_launch.py
│   ├── package.xml
│   ├── resource
│   │   └── robot_control
│   ├── robot_control
│   │   ├── __init__.py
│   │   ├── keyboard_control.py
│   │   ├── motor_control.py
│   │   └── teleop_example.py
│   ├── setup.cfg
│   ├── setup.py
│   └── test
│       ├── test_copyright.py
│       ├── test_flake8.py
│       └── test_pep257.py
└── robot_vision
    ├── CMakeLists.txt
    ├── config
    │   └── camera_parameter.yaml
    ├── include
    │   └── robot_vision_camera
    ├── launch
    │   └── robot_vision_camera.launch.py
    ├── package.xml
    └── src
        └── robot_vision_camera.cpp
```

Robot Control of Waverover (ROS2)

```python
def on_press(self, key):
    """press key callback"""
    try:
        if hasattr(key, "char") and key.char:
            c = key.char.lower()
            self.pressed_keys.add(c)

            if c == 'w':
                self.current_L = 0.3
                self.current_R = 0.3
            elif c == 's':
                self.current_L = -0.3
                self.current_R = -0.3
            elif c == 'a':
                self.current_L = -0.5
                self.current_R = 0.5
            elif c == 'd':
                self.current_L = 0.5
                self.current_R = -0.5
            elif c == 'x':
                self.current_L = 0.0
                self.current_R = 0.0
                self.pressed_keys.clear()

    except Exception as e:
        self.get_logger().error(f'Error in on_press: {str(e)}')
```

keyboard_control.py

```python
def motor_command_callback(self, msg):
    """receive motor commands from topic"""
    try:
        if len(msg.data) < 2:
            self.get_logger().warn('Motor command array must have at least 2 elements [L, R]')
            return

        L = float(msg.data[0])
        R = float(msg.data[1])

        # clip values to valid range
        L = np.clip(L, -0.5, 0.5)
        R = np.clip(R, -0.5, 0.5)

        # update command with thread safety
        with self.command_lock:
            self.motor_command = {"T": 1, "L": L, "R": R}
            self.last_command_time = time.time()

            # log if we're recovering from timeout
            if self.timeout_active:
                self.get_logger().info('Motor commands resumed - exiting safety timeout')
                self.timeout_active = False

        self.get_logger().debug(f'Motor command received: L={L:.2f}, R={R:.2f}')

    except Exception as e:
        self.get_logger().error(f'Error processing motor command: {str(e)}')
```

motor_control.py

Launch Camera

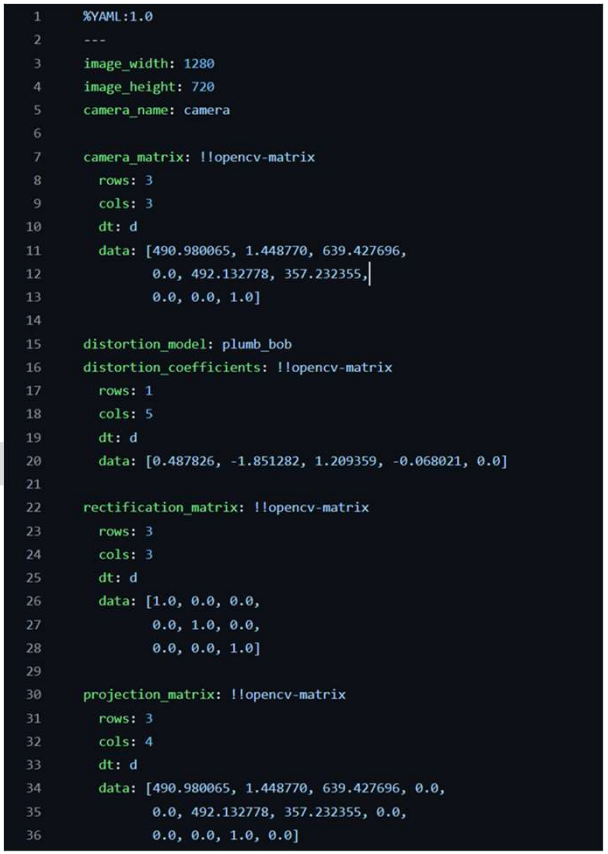
- Run camera driver nodes (link)

```
$ ros2 run robot_vision_camera robot_vision_camera #Camera Driver
```

- Or (link)

```
$ ros2 launch robot_vision_camera robot_vision_camera.launch.py #This included camera driver and foxglove bridge
```

- Camera Parameter



```yaml
%YAML:1.0
---
image_width: 1280
image_height: 720
camera_name: camera

camera_matrix: !!opencv-matrix
  rows: 3
  cols: 3
  dt: d
  data: [490.980065, 1.448770, 639.427696,
         0.0, 492.132778, 357.232355,
         0.0, 0.0, 1.0]

distortion_model: plumb_bob
distortion_coefficients: !!opencv-matrix
  rows: 1
  cols: 5
  dt: d
  data: [0.487826, -1.851282, 1.209359, -0.068021, 0.0]

rectification_matrix: !!opencv-matrix
  rows: 3
  cols: 3
  dt: d
  data: [1.0, 0.0, 0.0,
         0.0, 1.0, 0.0,
         0.0, 0.0, 1.0]

projection_matrix: !!opencv-matrix
  rows: 3
  cols: 4
  dt: d
  data: [490.980065, 1.448770, 639.427696, 0.0,
         0.0, 492.132778, 357.232355, 0.0,
         0.0, 0.0, 1.0, 0.0]
```

AprilTag Detection

- Run Apriltag detection node

```
$ ros2 run apriltag_ros apriltag_node --ros-arg
    -r image_rect:=/camera/image_raw \
    -r camera_info:=/camera/camera_info \
    --params-file
```

- Or (link):

```
$ ros2 launch apriltag_ros apriltag_launch.py
```

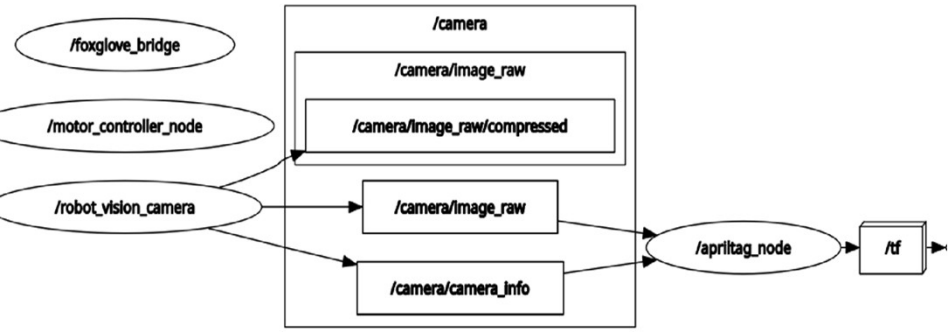
```yaml
/**:
    ros__parameters:
        image_transport: raw    # image format
        family: 36h11           # tag family name
        size: 0.165             # tag edge size in meter
        max_hamming: 1          # maximum allowed hamming distance (corrected bits)

        # see "apriltag.h" 'struct apriltag_detector' for more documentation on these optional parameters
        detector:
            threads: 4          # number of threads
            decimate: 2.0       # decimate resolution for quad detection
            blur: 0.0           # sigma of Gaussian blur for quad detection
            refine: True        # snap to strong gradients
            sharpening: 0.4     # sharpening of decoded images
            debug: False        # write additional debugging images to current working directory


        pose_estimation_method: "pnp"   # method for estimating the tag pose

        # optional list of tags
        tag:
            ids: [0, 1]           # tag ID
            frames: [tag_0, tag_1]  # optional frame name
            sizes: [0.165, 0.165]   # optional tag-specific edge size
```

AprilTag Detection





Reference

Waverover robot: link

Rubik Pi 3: link

ROS2 Jazzy: link

Rubik Pi 3 AP Mode: link

Questions?